

LAVA: Log ratios of Ancestral Variances

A Method for Identifying Local Adaptation in Structured Populations

Isabela do O, Oscar E Gaggiotti, Pierre de Villemereuil, Jerome Goudet

November 11, 2025

Abstract

The LAVA package implements the LogAV method for testing local adaptation in structured populations by comparing log ratios of ancestral variances. The method addresses limitations of traditional Q_{ST} - F_{ST} comparisons by accounting for complex population structures through coancestry matrices. The goal of the LAVA package is to provide a suggestion on how to implement the LogAV algorithm. This vignette provides a comprehensive guide to using LAVA, including theoretical background, practical examples, and interpretation of results.

Contents

1	Introduction	2
2	Theoretical Background	2
2.1	The LogAV Method	2
2.2	Population structure description	2
2.3	Coancestry Estimation	2
2.3.1	Population Coancestry Matrix (Θ_p)	2
2.3.2	Individual Coancestry Matrix (M)	2
2.3.3	Relatedness Adjustment	3
2.4	Mixed Effects Model	3
3	Installation and Setup	3
4	Data Requirements	3
4.1	Data Format	4
5	General step-by-step	4
5.1	Step 1: Prepare Your Data	4
5.2	Step 2: Calculate Coancestries	4
5.3	Step 3: Run LogAV Analysis	5
5.4	Step 4: Interpret Results	5
6	Tutorial: 18 population Stepping stones	5
6.1	Quick description of our example	5
6.2	Step 1: Getting the genetic data to dosages	6
6.3	Step 2: Loading trait data and population identification	7
6.4	Step 3: Calculating coancestries	7

6.5	Step 4: Visualizing coancestries	8
6.6	Running lava()	8
6.7	Advanced usage for posterior distribution	9
7	Interpreting results	10
7.1	Understanding the Log Ratio	10
8	Parameter description	10
8.1	calculate_coancestries	10
8.2	lava	11
9	Further Usage	12
9.1	Using Pedigree Information	12
9.2	Multiple Traits Analysis	12
10	Troubleshooting	13
10.1	Common Issues	13
10.1.1	Matrix Not Positive Definite	13
10.1.2	Population Mismatch Warnings	13
10.1.3	Convergence Issues	13
11	Best Practices	13
11.1	Experimental Design Recommendations	13
11.1.1	Sampling Strategy	13
11.1.2	What we found in our tests	14
11.2	Data Quality Checks	14
11.3	Pick your prior	14
11.4	Common questions	15
12	Limitations and Considerations	15
12.1	Assumptions	15
12.2	Potential Issues	15
13	Getting Help	15
A	Example usage with real data	16
B	Example Output	16

1 Introduction

Species are distributed across heterogeneous environments, leading to population divergence through genetic drift, natural selection, and limited gene flow. Distinguishing adaptive divergence from neutral processes is a fundamental challenge in evolutionary biology. Traditional methods like Q_{ST} - F_{ST} comparison assume equal relatedness among subpopulations—a simplification that can cause increase in false positive rates [cite de Villemereuil 2020, and do O 2025].

The LAVA package implements the LogAV method [cite do O 2025], which uses estimates of between- and within-population relatedness to model population structure. Under neutrality, the ancestral additive genetic variances estimated from between-population and within-population effects are expected to be equal.

2 Theoretical Background

2.1 The LogAV Method

LogAV tests the null hypothesis of neutral divergence by comparing two estimates of the ancestral additive genetic variance (V_A):

- $\hat{V}_{A,B}$: estimated from between-population effects
- $\hat{V}_{A,W}$: estimated from within-population effects

Under neutrality: $V_{A,B} = V_{A,W}$

The test statistic is:

$$\text{LogAV} = \log \left(\frac{\hat{V}_{A,B}}{\hat{V}_{A,W}} \right)$$

2.2 Population structure description

2.3 Coancestry Estimation

We use and recommend the usage of the allele-sharing method of moments [cite Weir and Goudet 2017 and Goudet and Weir 2023] to estimate coancestries for using the LAVA package.

LogAV uses two key matrices:

2.3.1 Population Coancestry Matrix (Θ_p)

Θ_p contains expected coancestries between pairs of subpopulations relative to the ancestral population. Element $\Theta_p^{i,j}$ describes the probability of identity-by-descent (IBD) between random alleles from subpopulations i and j .

$$\hat{\Theta}_p = \frac{\bar{A}_s^P - \min(\bar{A}_s^P)}{1 - \min(\bar{A}_s^P)}$$

2.3.2 Individual Coancestry Matrix (M)

M describes relatedness between pairs of individuals within subpopulations, accounting for the shared ancestry already captured in Θ_p .

2.3.3 Relatedness Adjustment

Within-population relatedness is adjusted to considering between-population coancestry twice as:

$$\hat{M}_x = \text{relatedness}_x \times (1 - \hat{\Theta}_x^p)$$

where $\hat{\Theta}_x^p$ is the diagonal element of $\hat{\Theta}_p$ for population x .

2.4 Mixed Effects Model

The phenotypic model is:

$$z \sim \mu + a_p + a_i + \epsilon$$

where:

$$a_p \sim N(0, 2\Theta_p V_{A,B}) \quad (1)$$

$$a_i \sim N(0, M V_{A,W}) \quad (2)$$

3 Installation and Setup

```
1 # Install from GitHub
2 devtools::install_github("isadoo/LAVA")
3
4 # It automatically installs required dependencies:
5 # - kinship2 (for kinship calculations)
6 # - hierfstat (for coancestry estimation)
7 # - brms (for Bayesian mixed models)
8 # - JGTeach (for population genetics utilities)
9 # - gaston (for genetic data manipulation)
10 # - Matrix (for efficient matrix operations)
11
12 library(LAVA)
```

4 Data Requirements

LAVA requires:

1. **Phenotypic data:** Offspring (F1) traits measured in common garden
2. **Genetic markers:** Neutral markers from founder population (non-common gardens)
3. **Relatedness for F1:** Pedigree information or F1 genetic markers (common gardens individuals)
4. **Population identification:** Founder and F1 individuals should be linked to a population.

4.1 Data Format

- Genetic data should be in dosage format (0, 1, 2 for diploids), with individuals as rows and loci as columns.
- Population identification for F1 should be a data frame including population names and individual names.
- Population identification for founder population should be a vector of population names.
- Trait data should be a data frame with at least two columns, one with individual identification and another with trait values.

5 General step-by-step

Here we show an abstract step-by-step of how to run the script. On the following section we provide a tutorial to be followed with the example dataset.

5.1 Step 1: Prepare Your Data

```
1 # Example data structure
2 head(genetic_data_parents)
3 #   ind_id  pop_id  locus1  locus2  locus3  ...
4 #   P001    pop1    1      0      2
5 #   P002    pop1    0      1      1
6 #   P003    pop2    2      2      0
7
8 head(trait_data)
9 #   id      trait_value
10 #   F001    15.2
11 #   F002    14.8
12 #   F003    16.1
13
14 head(population_ind)
15 #   id      population_id
16 #   F001    pop1
17 #   F002    pop1
18 #   F003    pop1
19 #   F004    pop2
20 #   F005    pop2
```

5.2 Step 2: Calculate Coancestries

```
1 # Extract population IDs for parents
2 parent_populations <- genetic_data_parents$pop_id
3
4 # Calculate coancestry matrices
5 coancestries <- calculate_coancestries(
6   genetic_data_parents = genetic_data_parents[, -c(1:2)], # Remove ID
7   columns
8   genotyped_parent_populations = parent_populations,
```

```

8   genetic_data_F1 = genetic_data_F1[, -c(1:2)], # If available
9   population_individual_id = population_ind,
10  column_individual = "id",
11  column_population = "population_id"
12 )
13
14 # Extract matrices
15 Theta.P <- coancestries$Theta.P
16 The.M <- coancestries$The.M
17
18 # Examine matrix properties
19 dim(Theta.P) # Should be n_populations x n_populations
20 dim(The.M)   # Should be n_individuals x n_individuals

```

5.3 Step 3: Run LogAV Analysis

```

1 # Run LAVA analysis
2 result <- lava(
3   Theta.P = Theta.P,
4   The.M = The.M,
5   trait_dataframe = trait_data,
6   column_individual = "id",
7   column_trait = "trait_value"
8 )
9
10 # View results
11 print(result)

```

5.4 Step 4: Interpret Results

```

1 # Plot posterior distributions
2 plot(result, which = "both")
3
4 # Extract key statistics
5 cat("Mean log ratio:", result$log_ratio$mean_log_ratio, "\n")
6 cat("P-value:", result$log_ratio$p_value, "\n")
7 cat("95% CI:", result$log_ratio$log_ratio_ci_lower,
8     "to", result$log_ratio$log_ratio_ci_upper, "\n")

```

6 Tutorial: 18 population Stepping stones

Here we show a step by step example of how to run lava on a simple simulated data set.

6.1 Quick description of our example

The example dataset comes from a simulated metapopulation:

- **Founder populations:** 18 subpopulations in stepping-stone structure
- **Evolutionary history:** 3000 generations in environmental cline creating selection gradient

- **Breeding design:** Controlled common garden with a North Carolina II design (add reference)
 - Within-population crosses to preserve population structure
 - Each population split into sires and dams
 - Multiple offspring per mating pair
- **Trait architecture:** Additive genetic model
 - 100 quantitative trait loci (QTLs)
 - Effect size: 0.1 per allele
 - heritability: $h_e^2 = 0.8$
 - Trait centered and standardized

6.2 Step 1: Getting the genetic data to dosages

```

1 # Read data from package example files
2 neutral_file <- system.file("extdata", "neutral_data_g3000.dat", package =
  "LAVA")
3 quanti_file <- system.file("extdata", "quanti_trait_g3000.dat", package =
  "LAVA")
4
5 # Read the genetic data using hierfstat
6 sim <- hierfstat::read.fstat(fname = neutral_file)
7 sim_quanti <- hierfstat::read.fstat(fname = quanti_file)
8
9 # Examine the data structure
10 head(sim)
11 sim[1:5, 1:5]
12 #      Pop n1_l1 n1_l2 n1_l3 n1_l4
13 # 1      1     22     11     21     11
14 # 2      1     22     11     11     11
15 # 3      1     22     11     22     11
16 # 4      1     22     11     11     11
17 # 5      1     22     11     11     11
18
19 # Convert biallelic data to dosage format
20 dos <- hierfstat::biall2dos(sim[, -1]) # Remove population column
21 dos_quanti <- hierfstat::biall2dos(sim_quanti[, -1], diploid = TRUE)
22
23 # Extract population IDs
24 pop <- sim$Pop
25
26 # Examine dosage output
27 dos_quanti[1:5, 1:5]
28 #      q1_l1 q1_l2 q1_l3 q1_l4 q1_l5
29 # [1,]      2      0      2      2      0
30 # [2,]      2      0      1      1      0
31 # [3,]      2      0      2      2      0
32 # [4,]      2      0      2      0      0
33 # [5,]      1      0      2      2      0

```

```

34
35 # Load F1 ("common gardens" population) dosages
36 dos_F1only_neutral <- readRDS(
37   system.file("extdata", "vignette_dos_F1only_neutral.rds", package = "
38     LAVA")
39 )

```

6.3 Step 2: Loading trait data and population identification

```

1 # Load example data from package
2 trait_df_pop <- read.csv(
3   system.file("extdata", "vignette_trait_df_pop.csv", package = "LAVA")
4 )
5
6 population_individual_id_df <- read.csv(
7   system.file("extdata", "vignette_population_individual_id_df.csv",
8     package = "LAVA")
9 )
10
11 head(trait_df_pop)
12 #   individual      trait trait_id population
13 # 1           1 -1.415731         1           1
14 # 2           2 -1.549296         1           1
15 # 3           3 -1.492134         1           1
16 # 4           4 -1.637156         1           1
17 # 5           5 -1.181021         1           1
18 # 6           6 -2.038686         1           1
19
20 head(population_individual_id_df)
21 #   pop_id individual
22 # 1       1         200
23 # 2       1         201
24 # 3       1         202
25 # 4       1         203
26 # 5       1         204
27 # 6       1         205

```

6.4 Step 3: Calculating coancestries

```

1 # Note: 'dos' and 'pop' should come from your genetic data processing
2 # For this example, assume they exist from Step 1
3
4 coancestries_dosage <- calculate_coancestries(
5   genetic_data_parents = dos,
6   genotyped_parent_populations = pop,
7   genetic_data_F1 = dos_F1only_neutral,
8   population_individual_id = population_individual_id_df,
9   column_individual = "individual",
10  column_population = "pop_id",
11  all_parents_genotyped = TRUE
12 )

```

```

13 Theta.P <- coancestries_dosage$Theta.P
14 The.M <- coancestries_dosage$The.M
15
16
17 # The calculate_coancestries function now uses kinship2::kinship()
18 # internally for more accurate relatedness calculations when using
    pedigree data

```

6.5 Step 4: Visualizing coancestries

```

1 heatmap(Theta.P,
2         Rowv = NA,
3         Colv = NA)

```

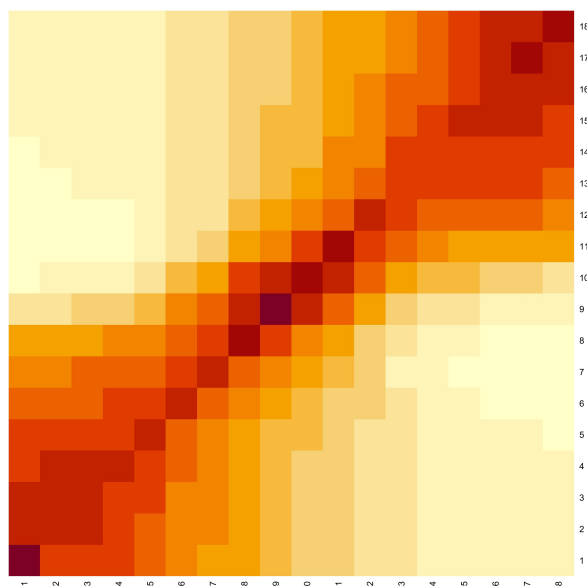


Figure 1: Θ^P heatmap

```

1 heatmap(The.M,
2         Rowv = NA,
3         Colv = NA)

```

6.6 Running lava()

```

1 results <- lava(Theta.P,
2                 The.M,
3                 trait_dataframe = trait_df_pop,
4                 column_individual = "individual",
5                 column_trait = "trait")
6
7 results

```

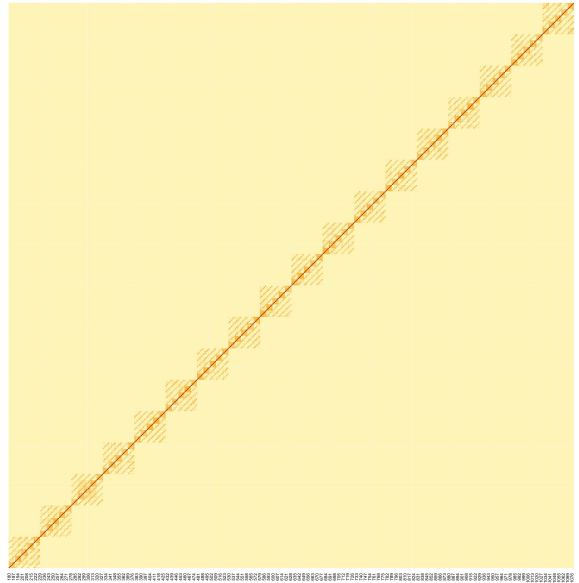


Figure 2: **M** matrix heatmap showing individual-level relatedness adjusted for population structure

```
# =====
# Log Ancestral Variance Analysis (LAVA)
# =====

# Log ratio of estimated ancestral variances (between equation/ # within equation):
  Mean: 1.9245 (95% CI: 1.0749 to 2.8382)

Difference in variance components (between equation/ within equation):
  Mean: NaN
  Median: NA (95% CI: NA to NA)

Statistical tests:
  Hypothesis test p-value: 0.0000

=====
```

6.7 Advanced usage for posterior distribution

We keep the full model as an attribute of the output.

```
1 #Access the brms model which was stored as attribute of lava's results
2 fit <- attr(results, "model")
3
4 #Now you can use it like any brms object
5 library(brms)
6 post_all <- as_draws_df(fit) #raw posterior samples
7 pp_check(fit) #checkign posterior samples
8 summary(fit)
```

```

Family: gaussian
Links: mu = identity; sigma = identity
Formula: Y ~ 1 + (1 | gr(pop, cov = two.Theta.P)) + (1 | gr(ind, cov = The.M))
Data: dat (Number of observations: 1000)
Draws: 8 chains, each with iter = 3000; warmup = 1000; thin = 2;
       total post-warmup draws = 8000

```

Multilevel Hyperparameters:

```

~ind (Number of levels: 1000)
      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)    0.34    0.03    0.28    0.40 1.00    1459    2500

```

```

~pop (Number of levels: 20)
      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)    0.90    0.19    0.60    1.33 1.00    5560    6842

```

Regression Coefficients:

```

      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept    0.07    0.46   -0.84    0.97 1.00    6596    6642

```

Further Distributional Parameters:

```

      Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma    0.33    0.02    0.29    0.36 1.00    1445    2010

```

7 Interpreting results

7.1 Understanding the Log Ratio

- $\text{LogAV} = 0$: Neutral evolution (equal ancestral variances)
- $\text{LogAV} > 0$: Evidence for local adaptation ($V_{A,B} > V_{A,W}$)
- $\text{LogAV} < 0$: Evidence for spatially-homogeneous selection ($V_{A,B} < V_{A,W}$)

8 Parameter description

8.1 calculate_coancestries

```

1 calculate_coancestries <- function(genetic_data_parents,
2                                   genotyped_parent_populations,
3                                   genetic_data_F1 = NA,
4                                   population_individual_id = NA,
5                                   column_individual = "id",
6                                   column_population = "population_id",
7                                   pedigree = NA,
8                                   all_parents_genotyped = FALSE)

```

- **genetic_data_parents**: A data frame containing genetic data for the parental generation in dosage format.

- `genotyped_parent_populations`: A vector of population IDs for the parental generation.
- `genetic_data_F1`: (Optional) A data frame containing genetic data for the F1 generation in dosage format. Default is NA.
- `population_individual_id`: (Optional) A data frame mapping individuals to their respective populations. Should contain parent IDs first, then F1 IDs. If not provided, a balanced design is assumed. Default is NA.
- `column_individual`: The name of the column containing individual IDs in the population mapping data frame. Default is "id".
- `column_population`: The name of the column containing population IDs in the population mapping data frame. Default is "population_id".
- `pedigree`: (Optional) A data frame containing pedigree information with at least three columns: individual ID, dam ID, and sire ID. Default is NA.
- `all_parents_genotyped`: Logical; if TRUE, indicates that the genotyped individuals in `genotyped_parent_populations` are the parents of F1 individuals. Default is FALSE.

8.2 lava

```

1  lava <- function(Theta.P,
2                    The.M,
3                    trait_dataframe,
4                    column_individual = "id",
5                    column_trait = "trait",
6                    formula = "Y ~ 1 + (1 | gr(pop, cov = two.Theta.P)) + (1 |
                                gr(ind, cov = The.M))", ...)

```

- `Theta.P`: A square matrix representing the coancestry matrix between populations.
- `The.M`: A square matrix representing the kinship-based relatedness matrix for individuals.
- `trait_dataframe`: A data frame containing individual IDs and trait values. The first column should be individual IDs, and the second column should be trait values.
- `column_individual`: The name of the column containing individual IDs in the trait data frame. Default is "id".
- `column_trait`: The name of the column containing trait values in the trait data frame. Default is "trait".
- `formula`: A formula specifying the model structure for the Bayesian mixed-effects model. Default is "Y ~ 1 + (1 | gr(pop, cov = two.Theta.P)) + (1 | gr(ind, cov = The.M))".
- `...`: Additional arguments passed to the `brms` function for model fitting.

9 Further Usage

9.1 Using Pedigree Information

```
1 # When pedigree is available, LAVA uses kinship2::kinship() internally
2 # for accurate relatedness calculations
3 pedigree_data <- data.frame(
4   id = c("F001", "F002", "F003"),
5   dam = c("P001", "P002", "P003"),
6   sire = c("P004", "P005", "P006"),
7   dam_pop = c("pop1", "pop1", "pop2"),
8   sire_pop = c("pop1", "pop1", "pop2")
9 )
10
11 coancestries_ped <- calculate_coancestries(
12   genetic_data_parents = genetic_data_parents[, -c(1:2)],
13   genotyped_parent_populations = parent_populations,
14   pedigree = pedigree_data,
15   all_parents_genotyped = TRUE
16 )
17
18 # The kinship2 package provides robust handling of:
19 # - Parent-offspring relationships (relatedness = 0.5)
20 # - Full sibling relationships (relatedness = 0.5)
21 # - Half sibling relationships (relatedness = 0.25)
22 # - Complex pedigree structures
23 # - Inbreeding coefficients
```

9.2 Multiple Traits Analysis

```
1 # Analyze multiple traits
2 traits <- c("body_length", "head_width", "fin_length")
3 results_list <- list()
4
5 for (trait in traits) {
6   results_list[[trait]] <- lava(
7     Theta.P = Theta.P,
8     The.M = The.M,
9     trait_dataframe = multi_trait_data,
10    column_individual = "id",
11    column_trait = trait
12  )
13 }
14
15 # Compare results
16 sapply(results_list, function(x) x$log_ratio$p_value)
```

10 Troubleshooting

10.1 Common Issues

10.1.1 Matrix Not Positive Definite

```
1 # Check eigenvalues
2 eigenvalues <- eigen(The.M)$values
3 if(any(eigenvalues < 0)) {
4   cat("Minimum eigenvalue:", min(eigenvalues), "\n")
5   # Consider data quality or increase sample size
6 }
```

10.1.2 Population Mismatch Warnings

Ensure population IDs are consistent across all data sources. And ensure that the M matrix is organized by population, with individuals of the same population next to each other.

10.1.3 Convergence Issues

```
1 # Increase iterations for better convergence
2 result <- lava(
3   Theta.P = Theta.P,
4   The.M = The.M,
5   trait_dataframe = trait_data,
6   iter = 4000,
7   warmup = 2000,
8   chains = 4
9 )
```

11 Best Practices

11.1 Experimental Design Recommendations

When designing your own common garden experiment for LAVA analysis, consider the following:

11.1.1 Sampling Strategy

1. Founder Population Sampling

- Collect samples from as many subpopulations as possible, as this increases power
- Ensure adequate sample size per population (Based on our simulations we suggest min 6 individuals per population)
- If possible add environmental information to increase power

2. Crossing Design

- *Within-population crosses*: We suggest within-population crosses to preserve population structure

- We have tested North Carolina II design with 5×5 sires and dams per population and 3×3
- We have also tested half-sib design with 9 sires and 1 dam per population.
- Record complete pedigree information (essential if not genotyping F1)
- Balance number of offspring per cross when possible.

3. Common Garden Setup

- Standardize environmental conditions across all F1 individuals
- Randomize individual positions to minimize spatial effects
- Control for maternal effects (e.g., egg size) if relevant
- Measure traits at consistent developmental stages

4. Genetic Data Collection

- *Option 1:* Genotype F1 at neutral markers for kinship estimation
- *Option 2:* Use pedigree information
- Ensure sufficient marker density for accurate coancestry estimation

11.1.2 What we found in our tests

Here we can add a little summary of the main results of our LAVA testing

11.2 Data Quality Checks

Before running LAVA, verify:

- No major genotyping errors or missing data patterns
- Population structure in founder data matches expectations
- F1 individuals cluster appropriately by parental population
- Trait distributions are reasonable (check for outliers, data entry errors)
- Matrix positive definiteness (LAVA will warn if issues detected). A very non-pd matrix is an indication of potential issues on the data, for example, individuals misplaced.

11.3 Pick your prior

As it should be clear by now, the `lava()` function uses Bayesian inference through the `brms` package, and is through it that the ancestral variances are estimated. Thus, it requires you to specify a **prior**; the probability distribution that will describe the expectations about your parameters. We intentionally do not set out default priors because it requires knowing your data. Among several other examples, we recommend you to consider: (1) The characteristics of the type of trait you are studying, whether it is continuous or categorical (*e.g.* height, mass, etc), if your trait is bounded (*e.g.* survival), or if there is some skewness (*e.g.* clutch size), etc. If you do not explicitly define your prior, it will use `brms` defaults. This first aspect of your data will be very important to choose the shape of the distribution you expect. (2) The measurement precision, sample sizes, common garden quality, will describe how large you expect your residual to be.

11.4 Common questions

1. Can the same dosage data be used to calculate $\mathbf{M}$ and Θ^P ? Ideally not. Θ^P should describe the relatedness the neutral structure of the populations we are comparing. When we expose F1 to our common gardens we are likely reducing the effect of selection and increasing the effects of drift from separating the populations and from introduced sampling variance. We note on our discussion, there are ways to reconstruct the parental genotypes and thus get Θ^P from only sampling genetic data of the common gardens populations. And as reviewer 2 pointed out given reliable ways of estimating the amount of drift between the founder population and the common gardens generation, the issue would be ameliorated
2. Is it necessary to use common gardens? We advise to always use a strategy to standardize the environmental effects over phenotypic measurements. Similarly to previous methods including $Q_{ST}-F_{ST}$, our method will be impacted by confounding effects of the environment and genotype-by-environment interactions.

12 Limitations and Considerations

12.1 Assumptions

- Panmictic ancestral population
- Additive genetic effects
- Common garden environment standardizes environmental effects
- Neutrality of the loci for estimating coancestry
- Sufficient SNP density for estimating coancestry

12.2 Potential Issues

I will add things here after the results we get from analyzing simulated data.

13 Getting Help

- Package documentation: `help(package = "LAVA")`
- Function help: `?lava`, `?calculate_coancestries`
- GitHub repository: <https://github.com/isadoo/LAVA>
- GitHub issues: <https://github.com/isadoo/LAVA/issues>
- Contact: isabela.doo@unil.ch
- Using kinship2 instead of our own function, see KINSHIP2_MIGRATION.md in the package documentation

A Example usage with real data

```
1 data("stickleback_example") # Hypothetical dataset
2
3 # Morphological traits
4 morphological_traits <- c("standard_length", "body_depth",
5                           "head_length", "pelvic_girdle_length")
6
7 # Behavioral traits
8 behavioral_traits <- c("aggression", "risk_taking")
9
10 # Analyze each trait
11 morph_results <- lapply(morphological_traits, function(trait) {
12   lava(Theta.P = stickleback_Theta.P,
13         The.M = stickleback_The.M,
14         trait_dataframe = stickleback_traits,
15         column_trait = trait)
16 })
17 names(morph_results) <- morphological_traits
18
19 # Summary table
20 results_table <- data.frame(
21   Trait = names(morph_results),
22   Mean_LogRatio = sapply(morph_results, function(x) x$log_ratio$mean_log_
23     ratio),
24   P_value = sapply(morph_results, function(x) x$log_ratio$p_value),
25   Significant = sapply(morph_results, function(x) x$log_ratio$p_value <
26     0.05)
27 )
28
29 print(results_table)
```

B Example Output

```
=====
Log Ancestral Variance Analysis (LAVA)
=====
```

Log ratio of estimated ancestral variances (between equation/ within equation):
Mean: 2.1483 (95% CI: 0.8542 to 3.4421)

Difference in variance components (between equation/ within equation):
Mean: 0.3421
Median: 0.3387 (95% CI: 0.1234 to 0.5612)

Statistical tests:
Hypothesis test p-value: 0.0285

```
=====
```